

RELATÓRIO DE PROJETO PRÁTICO: GERENCIADOR DE TIMES DE FUTEBOL

1. Introdução

O relatório a seguir descreve o projeto de criação do trabalho de IALG (Introdução aos algoritmos) -UFLA.

O sistema desenvolvido possibilita operações de CRUD (Create, Read, Update, Delete), permitindo ao usuário manipular uma base de dados de times de futebol, garantindo a integridade das informações. O projeto foi estruturado para ser facilmente executável, permitindo compilação e execução em ambientes Linux e Windows, conforme solicitado.

2. Descrição das Estruturas e Lógica do Programa

O sistema foi estruturado utilizando uma struct denominada "Time", que agrega os dados de cada registro. Também foram utilizadas algumas variáveis globais como "int tamanhoAtual" "int capacidade" "const string "NOME_ARQUIVO"

2.1 Estrutura de Dados

Para o armazenamento, foi utilizado um vetor alocado dinamicamente. A escolha por alocação dinâmica, em vez de vetores estáticos, deve-se pela necessidade de flexibilidade, permitindo o redimensionamento conforme o crescimento da base de dados. Quando o limite de capacidade do vetor é atingido, o programa realiza uma realocação somando 10 posições à capacidade anterior como pedido pela descrição do trabalho, garantindo eficiência na gestão da memória e evitando overflow.

2.2 Lógica de Operações

- **Leitura e Escrita:** O programa inicia chamando a função “carregarArquivo”. Função está especializada em abrir o arquivo “times.csv” e preencher o vetor times, esta função também é responsável por realocar o vetor caso seja necessário automaticamente, criando um vetor com o novo tamanho apagando o antigo e o substituindo pelo novo. Logo após é chamado a função “introducao”, função está responsável por abrir uma linha de texto para apresentação do programa. Em seguida é aberto o menu inicial do programa em que disponibiliza diversas opções para o usuário, Elas são:

1- Inserir novo time

2- Remover dados existentes

3- Buscar time

4- Lista completa

5- Exibir intervalo de registros

6- Ordenar registros

7- Alterar registro

8- Salvar alterações

0- Sair

- **Buscas:** Foram implementadas duas formas de busca. A busca sequencial, que percorre o vetor linearmente, e a busca binária. Esta última garante maior performance contudo ela necessita que o vetor esteja organizado devido a sua natureza de “Dividir e conquistar”. Por isto a função “buscaTimebinaria” organiza o vetor previamente antes de ser feita a busca.
- **Ordenação:** Utilizamos o método *Selection Sort* para organizar os registros por nome ou por identificador. A escolha deste método foi fundamentada em sua simplicidade de implementação e adequação ao volume de dados proposto. O Selection Sort funciona basicamente dividindo o vetor em duas partes sendo uma ordenada e outra não ordenada. A cada ciclo, ele busca o menor elemento na parte não ordenada e o troca com o primeiro elemento dessa lista, aumentando a parte ordenada em uma unidade. Dessa forma atendendo aos requisitos sem o uso de bibliotecas prontas como a “sort”.

3. Ordem dos Dados no Arquivo

O arquivo `times.csv` segue uma estrutura linear delimitada por pontos e vírgulas. A organização das colunas no arquivo é:

1. **Identificador** (Inteiro)
2. **Nome do Time** (String com espaços)
3. **Cidade de Origem** (String com espaços)
4. **Ano de Fundação** (Inteiro)
5. **Número de títulos (inteiro) contando apenas títulos internacionais e nacionais oficiais**

Esta estrutura possibilita que o programa realize a cada linha por vez, assim lendo cada trecho delimitado por ponto e vírgula por vez, garantindo que mesmo com campos em branco sejam lidos corretamente.

4. Análise Crítica: Acertos e Erros

Durante o desenvolvimento gostaríamos de destacar:

- A modularização do código em arquivos `.h` e `.cpp`, que facilitou a organização do projeto e a manutenção do código-fonte, não se limitando a deixar todo o programa em um único arquivo `.cpp`.
- A implementação da remoção lógica (marcação com chave negativa), o que facilitou a separação e futura exclusão de arquivos desejados, bem como ignorar os mesmos anteriormente a sua remoção.

Durante o desenvolvimento foram notadas as seguintes dificuldade-erros do programa

Como **dificuldades**, A principal dificuldade foi a leitura de strings com espaços na leitura do csv, para solução foi necessário utilizar a função getline com o delimitador de ; . Outra grande dificuldade foi com relação ao redimensionamento do vetor, no qual foram exigidos diversos testes no código e modificações ate o mesmo funcionar sem erros.

Alguns erros do programa podem surgir como inserir uma entrada char ou string no lugar de uma int ou vice-versa, implementamos a limpeza do buffer de entrada, evitando travamentos no ciclo principal de execução. Contudo em algumas areas em especifica o erro de entrada pode quebrar a execução do programa, sendo necessário reinicialização do mesmo. Outro erro frequente era falta de ordenação caso o usuário escolhesse o menu buscar time e escolher o método de busca binaria. Portanto foi necessária a realização de ordenação previamente a busca, embora o mesmo processo gaste poder computacional desnecessário caso o arquivo já esteja ordenado.

5. Conclusão e Resultados

O projeto permitiu aplicar de forma prática grande parte do conteúdo passado em aulas até o momento na disciplina de introdução a computação e introdução aos algoritmos. O sistema cumpriu todos os requisitos solicitados: realiza o carregamento, redimensionamento eficiente, busca rápida e ordenação correta dos registros.

Os testes realizados demonstram que o sistema é estável e eficiente para o volume de dados proposto (acima de 100 itens). Gostaríamos de ressaltar que a experiência de desenvolver um sistema "do zero", sem o auxílio de bibliotecas de que faziam estas funções de maneira simplificada foi um grande desafio, contudo após pesquisas nos materiais de estudos disponibilizados e fontes online foi possível criar um programa "robusto" e que atende todos os requisitos.